

Final Capstone Project Guidelines

Industry-Oriented Data Structures & Algorithms Using C

1. Project Overview

The **Final Capstone Project** is the culminating assessment for the **Data Structures & Algorithms using C** course. Students are expected to design and develop a **console-based industry-oriented application** by applying programming fundamentals, advanced data structures, algorithm design, problem-solving techniques, and software engineering best practices.

The project should demonstrate:

- Strong understanding of Data Structures & Algorithms
 - Modular programming in C
 - Efficient memory management
 - Problem-solving and optimization
 - File handling and persistent storage
 - Git & GitHub version control
 - Technical documentation
 - Professional project presentation
-

2. Project Objectives

Upon successful completion of the project, students should be able to:

- Apply appropriate data structures to solve real-world problems.
- Analyze algorithm efficiency using Big O notation.
- Implement modular and reusable C programs.
- Manage dynamic memory safely using pointers.
- Build an end-to-end console application.
- Follow industry coding standards.
- Maintain source code using GitHub.
- Present technical solutions professionally.

3. Suggested Project Domains

Students may select **any one** of the following domains or propose an equivalent project with faculty approval.

Project	Recommended Data Structures
Student Management System	Arrays, Linked List, Searching, Sorting
Employee Record Management	Arrays, Linked List, File Handling
Library Management System	Linked List, Stack, Queue, BST
Hospital Management System	BST, Queue, Hashing
Banking Management System	Linked List, Queue, Searching
Inventory Management System	Hashing, Sorting, BST
Browser History Manager	Stack, Doubly Linked List
Task Scheduler	Queue, Priority Queue
Social Network Simulation	Graph, BFS, DFS
Mini Search Engine	Hashing, Trees
Navigation System	Graph, Dijkstra Algorithm

4. Minimum Technical Requirements

Each project should demonstrate implementation of **at least eight** major concepts from the syllabus.

Topic	Mandatory
Structures	✓

Functions	✓
Dynamic Memory Allocation	✓
File Handling	✓
Searching Algorithm	✓
Sorting Algorithm	✓
Linked List / Stack / Queue	✓
Trees / Graph / Hashing (at least one)	✓
Recursion	✓
Complexity Analysis	✓

5. Functional Requirements

Every project should include the following core modules.

Module	Description
Authentication/Menu	User-friendly menu-driven application
CRUD Operations	Add, Update, Delete and Display records
Searching	Efficient search implementation
Sorting	Display sorted records
File Storage	Persistent data storage
Reports	Summary statistics
Error Handling	Input validation and exception handling
Exit Module	Safe termination with data saving

6. Sample User Stories

User Stories

User Story ID	User Story	Priority	Acceptance Criteria
US-01	As an Administrator, I want to add a new record so that new information can be stored in the system.	High	Record is validated and stored successfully.
US-02	As a User, I want to search records using an identifier so that I can retrieve information quickly.	High	Correct record is displayed when available.
US-03	As an Administrator, I want to update existing records so that incorrect information can be corrected.	High	Existing data is updated successfully.
US-04	As an Administrator, I want to delete records so that obsolete information is removed.	Medium	Selected record is deleted without affecting other records.
US-05	As a User, I want to display all records in a formatted manner so that information is easy to read.	High	Records are displayed in tabular format.
US-06	As a User, I want data stored in files so that information is available after restarting the application.	High	Records are saved and loaded correctly.
US-07	As a Developer, I want to use linked lists so that dynamic insertion and deletion are efficient.	High	Dynamic memory allocation is implemented successfully.
US-08	As a User, I want undo functionality so that accidental operations can be reversed.	Medium	Last operation can be restored using a stack.
US-09	As a User, I want requests processed in order so that fairness is maintained.	High	Queue follows FIFO order.

US-10	As a User, I want records sorted before display so that information is organized.	Medium	Sorting algorithm produces correct ascending/descending output.
US-11	As a Developer, I want recursive functions where appropriate so that solutions are modular.	Medium	Recursive implementation executes correctly.
US-12	As a User, I want records stored in a Binary Search Tree so that searching is efficient.	High	BST operations work correctly.
US-13	As a User, I want fast lookup using hashing so that search performance improves.	High	Hash table retrieves records correctly.
US-14	As a User, I want related entities connected using graphs so that relationships can be visualized.	Medium	Graph traversal using BFS/DFS is implemented.
US-15	As a Developer, I want algorithm performance analyzed so that optimization opportunities can be identified.	High	Time and space complexity are documented for major operations.

7. GitHub Repository Structure

Project-Name/

```
| — src/
| — include/
| — docs/
| — report/
| — presentation/
| — screenshots/
| — sample-data/
| — README.md
| — LICENSE
| — Makefile
| — main.c
```

8. GitHub Submission Requirements

Requirement	Description
Repository Name	Meaningful project name
README	Project overview, setup instructions and screenshots
Source Code	Well-structured and commented
Commit History	Regular commits showing project progress
Documentation	Complete project report
Presentation	Final presentation slides
Screenshots	Output screenshots
License	Open-source license (recommended)

9. Final Project Deliverables

Deliverable	Description	Format
Source Code	Complete working C project	GitHub Repository
README	Installation guide and project overview	Markdown
Project Report	Detailed technical documentation	PDF
Presentation	Final project presentation	PPT/PDF
GitHub Repository	Public repository with commit history	GitHub Link
Source Code Archive	Backup of complete project	ZIP
Output Screenshots	Screenshots of application execution	PNG/JPG
Test Cases	Functional test cases with expected results	PDF/Excel

Complexity Analysis	Time and Space Complexity of major operations	PDF
Demonstration Video (Optional)	Project walkthrough	MP4/YouTube Link

10. Project Report Structure

Chapter	Contents
Chapter 1	Introduction
Chapter 2	Problem Statement
Chapter 3	Requirement Analysis
Chapter 4	System Design
Chapter 5	Data Structures Used
Chapter 6	Algorithm Design
Chapter 7	Implementation
Chapter 8	Testing
Chapter 9	Complexity Analysis
Chapter 10	GitHub Repository
Chapter 11	Results & Discussion
Chapter 12	Conclusion & Future Scope
References	Books, Articles, Websites

11. Presentation Structure

Slide	Content
1	Project Title & Team Details

2	Problem Statement
3	Objectives
4	System Architecture
5	Functional Modules
6	Data Structures & Algorithms Used
7	Application Demonstration
8	Complexity Analysis
9	Challenges & Learning Outcomes
10	GitHub Repository
11	Future Enhancements
12	Questions & Answers

12. Project Evaluation Rubric (100 Marks)

Evaluation Criteria	Marks
Problem Understanding & Solution Design	10
Implementation of Data Structures	20
Algorithm Design & Optimization	15
Code Quality & Programming Standards	10
Memory Management & File Handling	10
GitHub Repository & Version Control	10
Documentation (Project Report)	10
Presentation & Demonstration	10
Viva / Technical Questions	5
Total	100

13. Final Completion Checklist

Item	Status (✓)
Project Completed	
Code Compiles Successfully	
No Runtime Errors	
README Updated	
GitHub Repository Submitted	
Minimum Eight DSA Concepts Implemented	
Project Report Submitted	
Presentation Submitted	
Screenshots Included	
Complexity Analysis Added	
Viva Completed	

14. Deliverables Required for Final Completion Certificate

The **Course Completion Certificate** will be issued only after all mandatory deliverables have been submitted, verified, and accepted.

S. No.	Deliverable	Mandatory	Verification Criteria
1	Working C Project	✓	Project compiles and executes successfully

2	Public GitHub Repository	✓	Repository accessible with complete source code and commit history
3	README Documentation	✓	Includes project overview, setup instructions, DSA concepts, and screenshots
4	Project Report	✓	Submitted in PDF format as per prescribed template
5	Presentation Slides	✓	10–12 slide presentation covering project implementation
6	Project Demonstration	✓	Live demonstration during final evaluation
7	Viva Voce	✓	Technical discussion with evaluator
8	Test Cases & Results	✓	Functional test cases with expected and actual outcomes
9	Complexity Analysis	✓	Time and space complexity documented for major operations
10	Source Code Quality	✓	Proper modularization, comments, naming conventions, and error handling
11	Faculty Evaluation	✓	Meets minimum passing criteria as per evaluation rubric

Note: Students must successfully complete **all mandatory deliverables** and obtain the minimum qualifying marks in the final evaluation to be eligible for the **Course Completion Certificate**. Failure to submit any mandatory deliverable may result in withholding of the certificate until all requirements are fulfilled.